Ranking Candidates the Way a Great Recruiter Would

Intelligent Candidate Discovery & Ranking Challenge — Approach Deck

Hybrid scoring system: TF-IDF semantic similarity + rule-based career/skill/behavioral features

Team: REPLACE_ME

The Problem

Keyword search is blind to fit

A Marketing Manager with 'RAG' tagged on their profile ranks the same as an engineer who actually built one.

Great fits hide in plain language

Real Tier-5 candidates often never say 'embeddings' or 'Pinecone' — the evidence is in what they built, described in their own words.

Static profiles lie about availability

A perfect resume that hasn't logged in for 6 months isn't actually hireable right now.

System Architecture

A hybrid pipeline: one vectorized semantic pass + five rule-based feature checks, combined into a weighted composite score.

Weighted composite score → sort → top 100 → CSV with reasoning

Semantic Similarity + Trust-Weighted Skills

weight: 0.20 + 0.25

We compare each candidate's career narrative (headline, summary, role descriptions) against the JD using TF-IDF + cosine similarity — a vector-space technique that catches semantic overlap even when exact keywords differ.

Skills are not counted by presence alone: each match is weighted by proficiency, months of actual use, and endorsements received, so a tagged-but-never-used skill contributes far less than one backed by real history.

Why this catches what keyword matching misses:

A candidate who built a 'recommendation system at scale' scores well on semantic similarity even if they never typed the word 'ranking.' A skill tag with 0 endorsements and 0 months used barely moves the score — defeating simple keyword stuffing.

Title & Career-History Fit

weight: 0.25

This component encodes the JD's explicit disqualifiers as direct rule checks against career history, not just the current title:

- Entire career at consulting firms only (TCS, Infosys, Wipro, etc.)
- Pure-research background with no evidence of production deployment
- Computer vision / speech / robotics background without NLP/IR exposure
- Non-technical current title (HR, Sales, Marketing, Support) even if skills are buzzword-heavy

Why this catches what keyword matching misses:

This is the direct counter to the JD's own stated trap: 'a candidate who has all the AI keywords listed as skills but whose title is Marketing Manager is not a fit.' A pure embedding-similarity model can be fooled by buzzword density; this rule layer cannot.

Experience, Location & Behavioral Modifier

weight: 0.10 + 0.05 + 0.15

Experience fit scores closeness to the JD's 5–9 year band (soft, not a hard cutoff, per the JD's own framing). Location fit favors Pune/Noida, with partial credit for other Indian metros and relocation-willing candidates.

The behavioral modifier combines login recency, recruiter response rate, open-to-work status, interview completion rate, and notice period into a single availability signal.

Why this catches what keyword matching misses:

The JD says it directly: 'a perfect-on-paper candidate who hasn't logged in for 6 months… is, for hiring purposes, not actually available.' This component is a modifier on top of fit — not a primary driver — matching exactly how the JD frames it.

Honeypot Detection

~80 candidates in the pool have subtly impossible profiles. Top-100 honeypot rate > 10% = disqualified.

- Years-of-experience vs. career-history-duration mismatch (ratio < 0.4 or > 2.5)
- 3+ skills marked 'expert' with ~0 months actually used
- Multiple simultaneously-current full-time jobs
- Invalid education date ordering
- Overlapping employment history across companies

Flagged when 2+ independent signals fire — an ensemble of weak heuristics, the same principle fraud-detection systems use. Honeypots in our top 100: 0.

Compute Constraints — Satisfied

~51s	0	0	0
Total runtime, 100K candidates	GPU calls	Network / hosted LLM calls during ranking	Honeypots in top 100

The semantic-similarity step is one vectorized TF-IDF + cosine-similarity matrix operation over all 100K candidates — not a per-candidate loop or API call. This is the architectural choice that makes the 5-minute, CPU-only, no-network budget achievable.

```
Reproduce with: python rank.py --candidates ./candidates.jsonl --out ./submission.csv
```

Honest Reasoning, Not Templated Filler

"Senior Machine Learning Engineer with 7.2 yrs; core skills: Weaviate, Recommendation Systems, Pinecone; technical title history; based in Noida, Uttar Pradesh"

Every reasoning string is built directly from the same scored features used to rank the candidate — never a separate, ungrounded text-generation step. Nothing is asserted that wasn't actually matched in the candidate's own data.

AI tools (Claude) were used for architecture discussion and code generation throughout development. All scoring logic and JD interpretation were reviewed and tuned by the team. No candidate data was sent to any external LLM API.